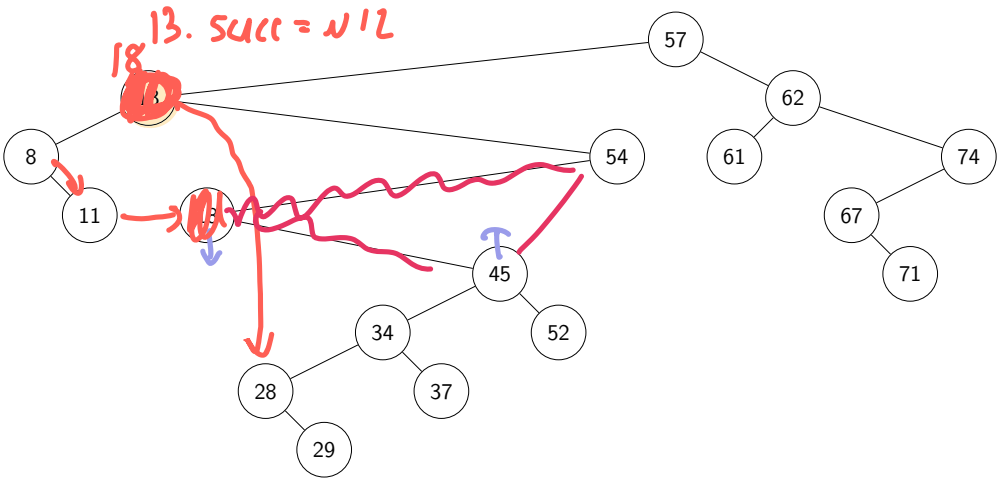
**Problem 1.** Consider the following binary search tree,

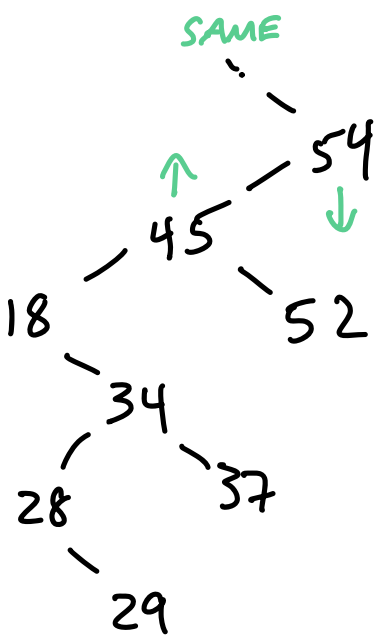


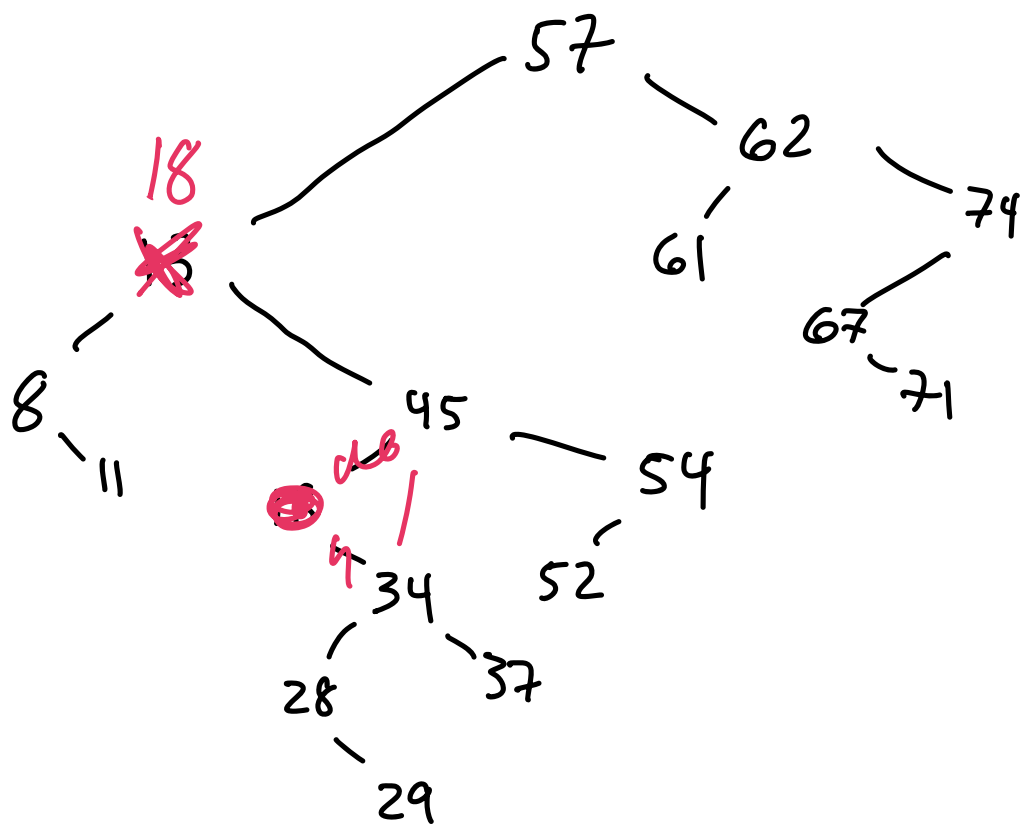

Perform the following operations

`LeftRotate`(18)

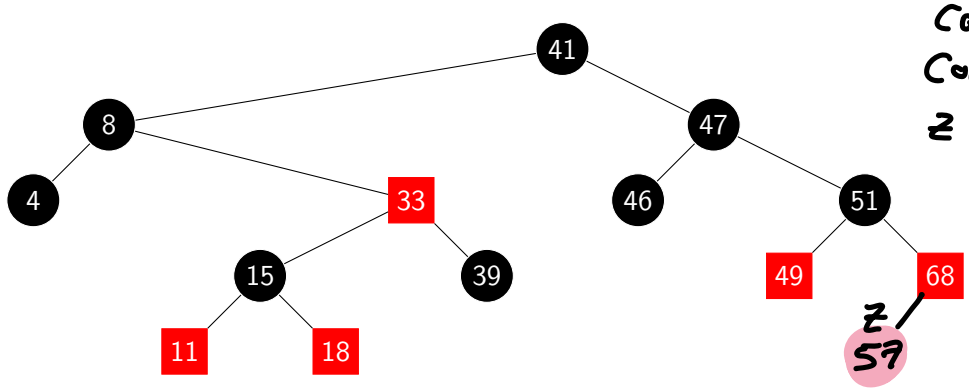
`RightRotate`(54)

`Delete`(13)



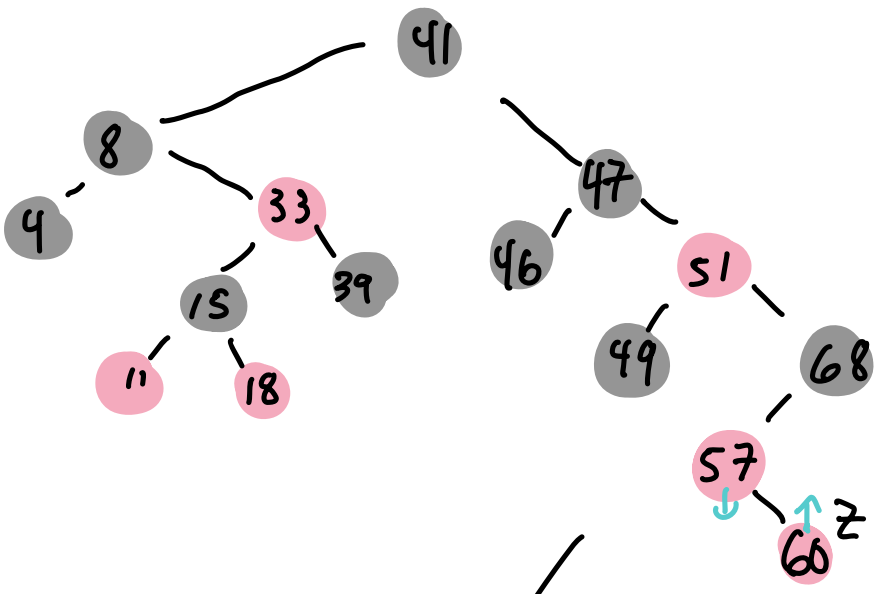
**Problem 2.** Given the red-black tree,




Case 1:
Color 49, 68 black
Color 51 red
z = 51

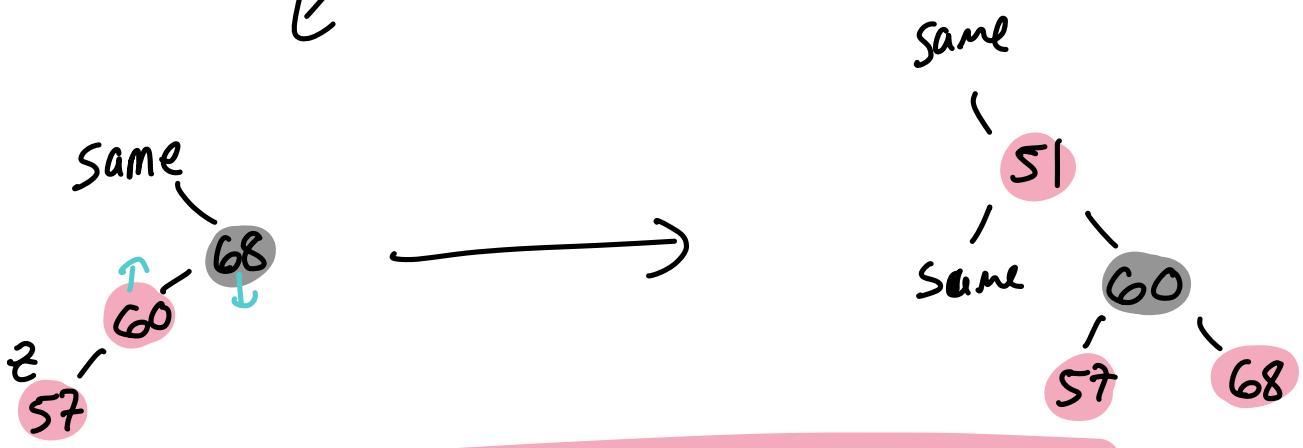
Perform the following sequence of operations in order.

1: RBT.Insert(57)
2: RBT.Insert(60)




case 2: LR(57)
z = 57

case 3: RR(68)
color 60 black
color 68 red




Insert(1) into empty tree

color root black

**Problem 3.** Given a Binary Search tree that has a pointer `successor` in every node, where $v$.`successor` points at the successor of the node $v$ (if $v$ has no successor, then $v$.`successor` = **NIL**).
Write a method called `DeleteAndUpdateSuccessor` that deletes a node from the tree and ensures that the `successor` pointer of every node in the tree is correct after the deletion.
Be sure to update as few nodes as possible.

function DeleteAndUpdate(T, z)
    Let p = predecessor of z
    Let S = successor of z
    Delete z from T
    p.successor = S

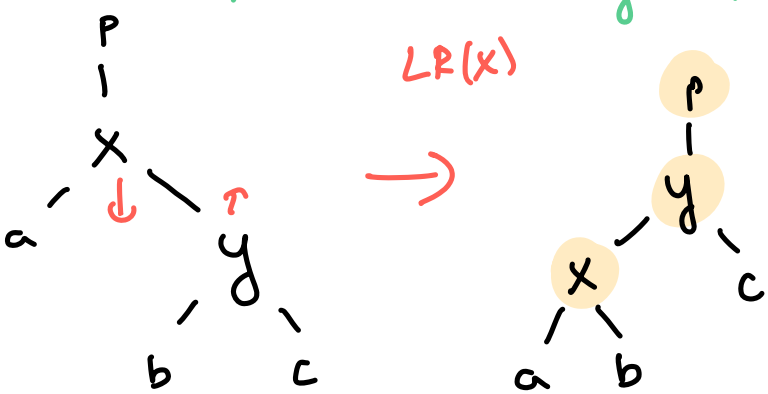
**Problem 4.** Given a Binary Search tree that has a pointer `product` in every node, where $v$.`product` stores the product of all of the keys in the subtree rooted at $v$ (including $v$).

Write a method called `LeftRotateAndUpdateProduct` that left rotates a node in the tree and ensures that the `product` field of every node in the tree is correct after the rotation.

Be sure to update as few nodes as possible.

Assume NIL.product=1

```
function LeftRotateAndUpdate(T, x)
    LeftRotate(T, x)
    x.product = x.key · x.left.product · x.right.product
    y = x.parent
    y.product = y.key · x.product · y.right.product
```

y.product = x.product

x.product = x.key · x.left.product · x.right.product